IS442 – Exam (Project Use) [20 marks]

General Instructions:

1. **You are not allowed to communicate in any way during the test. No mobile phones, earphones, smart watch or other AI / WIFI related devices. Any violation of this instruction will result in a zero score for your exam.**

2. You will perform the exam on your personal laptop. **Your screen will be recorded. IDE or Tools** that enable **AI** pair programming (e.g., GitHub co-pilot) or pair programming (Live Share) are **not allowed**. **Any violation of this instruction will result in a ZERO score.**

3. You can refer to any files on your laptop.

   Failure to do the following will trigger a penalty up to 20% of your score for that question.

4. Make sure your code can generate exactly the same output as we show in the sample runs. You may get some marks deducted for missing spaces, missing punctuation marks, misspelling, etc. in the output.

5. Do not hardcode. We will use different test cases to test and grade your solutions.
   a. Grading will be automated; i.e. no partial credit will be given for code.
   b. Grading will use a different set of test data to detect hardcoding.

6. Follow standard Java coding conventions (e.g. naming of getter and setter methods, choice of identifier names for classes, methods and variables) as well as indent your code correctly.

7. Ensure that all your Java code can compile without errors. They must compile with any test class(es) that are provided. You may wish to comment out the parts in your code, which cause compilation errors. But remember that commented code will NOT be graded. **Only code without compilation errors SHALL be graded.** You may need to comment out parts of the test class' code that you have not implemented in order to test your implemented solutions.

8. Download the source code and rename the root folder in the zip file to your **Email ID (e.g. `jiagu.wen.2025`)**

9. Include your name as author in the comments of all your submitted source files. For example, if your registered name is "Jia Gu Wen", include the following block of comments at the beginning of each source file (.java) you write.

```
/*
* Name     : Jia Gu Wen
* Email ID: jiagu.wen.2025
*/
```

10. Do **NOT**
    a. change the signature (parameter and return type) of the methods and DO NOT add in unnecessary packages.
    b. rename the file name of the Java files.

11. **You can write additional private (static or instance) helper methods and default access helper classes within the given .java files to solve the questions.**

Question 1 **[6 marks]**

In the folder <your email ID>\Q1. Insert your name and email ID to the top of
1. Q1a.java,
2. Q1b.java,

(a) **[Difficulty:*]** Implement the static method named `getIsogramWords`. **[3 marks]**
- takes in 1 parameter:
  - `inputs` (type: `ArrayList<String>`): This is a list of Strings.
- return
  - `ArrayList<String>` containing the Isogram words in `inputs`.
  - Isogram is a word whereby the characters within the words are unique
  - Assume that Strings in`inputs` will only contain alphabets.
  - Assume that `inputs` will not be null or an empty `ArrayList`
  - Assume that `inputs` will contain not null words
  - Assume that `inputs` will only contain empty String or single words

For example,
if `inputs` is ["Alan", "Ben", "Evelyne", "Danny"] , the return value is ["Ben"]
- Alan is not included as there are 2 A/a
- Evelyne is not included as there are 3 E/e
- Danny is not included as there are 2 n

Refer to Q1a.java for more test cases.

(b) **[Difficulty:*]** Implement the static method named `getSumOfEvenIntegers`. **[3 marks]**
- takes in 1 parameter:
  - `inputs` (type: `ArrayList<Object>`): This is a list of Objects. Not `null`.
- return
  - `int` which is the <u>sum</u> of all the <u>even integers</u> in `inputs`
  - Assume that `inputs` will not be null or an empty `ArrayList`

For example,
if `inputs` is [10, "a", 3, true, 20.0, 4] , the return value is 14
- 10 + 4 = 14 , the rest of the values are not an <u>even integer</u>

Refer to Q1b.java for more test cases

Note: **DO NOT HARDCODE** your methods. We will test your implemented methods using a different `main` method in which different values will be passed into these methods.

IMPORTANT : Ensure Q1a.java, Q1b.java
are in the folder <your email ID>/Q1
Only these files will be marked

Question 2 **[10 marks]**

In the folder <your email ID>\Q2. Insert your name and email ID to the top of
1. Q2a.java,
2. Q2b.java,

(a) **[Difficulty:*]** Implement the static method named `getAverageAge`. **[5 marks]**

- takes in 2 parameters:
  - `filename` (type: `String`): This is the name of the input file (e.g. persons.txt)
    - An entry in the file would have the following format
      - <Name>-<Age> e.g. John LEE-28
  - `surname` (type: `String`): This is the surname.

- return
  - `double` which is the value of the average age of the people in the input file where the surname matches with the parameter `surname`
  - Assume that a person's surname in the input file can only either be the first word or the last word in a name. (e.g. John LEE, LEE Teck Leong) and the surname are in uppercase.
  - Assume there will always be a surname for each person in the input file.
  - No rounding required for the return value
  - Returns -1.0 if the file doesn't exists.
  - Returns 0.0 if the file doesn't contain a person of that surname

Refer to Q2a.java for more test cases

(b) **[Difficulty:**]** Implement the static method named `getTopStudent`. **[5 marks]**

- takes in 2 parameters:
  - `filename` (type: `String`): This is the name of the input file. (e.g. students.txt)
    - An entry in the file would have the following format
    - <Name>,<Courses> e.g. John LEE,IS101#4.0-IS102#3.0-IS103#2.5
    - John LEE takes 3 courses (IS101 with GPA of 4.0, IS102 with GPA of 3.0, IS103 with GPA of 2.5)
    - The number of courses are dynamic for each student
  - `courseName` (type: `String`): This is the name of the course. (e.g. IS101)

- return
  - `String` which contains the name and gpa of the student and which has the highest GPA where the course matches with the parameter `courseName`
    e.g. John LEE-4.0
  - If there are multiple students having the highest GPA, take the first occurrence.
  - You are given `DataException.class`. `DataException` is a RuntimeException
  - Throws `DataException` if the file doesn't exists.
  - Throws `DataException` if the file doesn't contain a course of that coursename.

Refer to Q2b.java for more test cases.

Note: **DO NOT HARDCODE** your methods. We will test your implemented methods using a different `main` method in which different values will be passed into these methods.

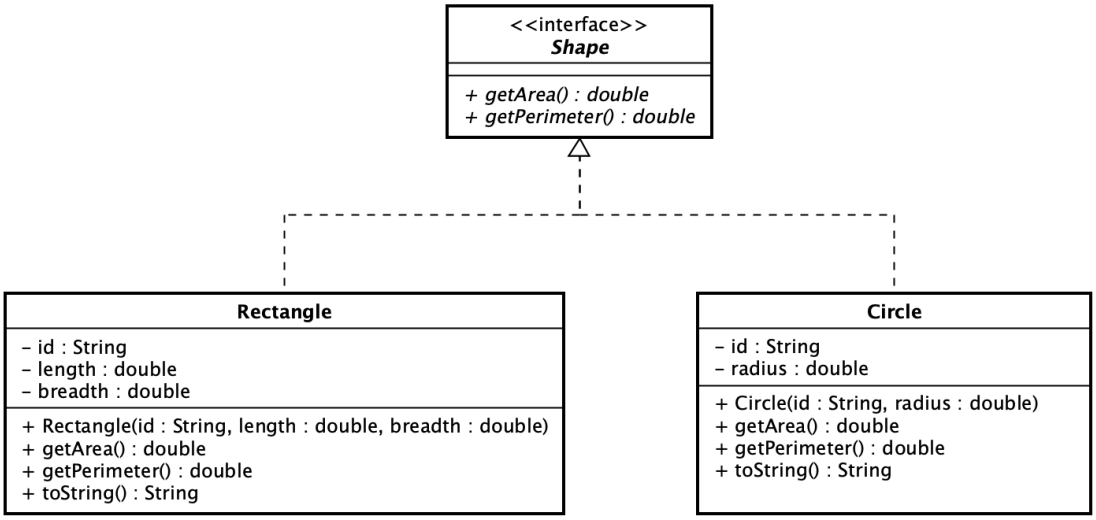
IMPORTANT : Ensure Q2a.java, Q2b.java
are in the folder <your email ID>/Q2
Only these files will be marked

Question 3 **[4 marks]**

In the folder <your email ID>\Q3. Insert your name and email ID to the top of
1. Q3.java,
2. ShapeComparator.java,

You are given the API documentation of the following Java classes and their byte code (.class) files. (The API documentation is located at the folder **<your email ID>\Q3\API\index.html** and the class files are inside **<your email ID>\Q3**). Study the API documentation of these classes.

- `Shape, Rectangle, Circle`



[Difficulty:]** In Q3.java, Implement the static method named `sortShapes`. **[4 marks]**

- takes in 1 parameter:
  - `shapeList` (type: `List<Shape>`): A list of `Shape` objects.

- return
  - a list of shapes which has area <= 1000 and sorted in the following order
    - area by descending manner
    - perimeter in descending manner.
  - You **MUST** complete the codes in ShapeComparator.java and use it in `sortShapes`.
  - Assume that `shapeList` will not be null or an empty `ArrayList`

Refer to Q3.java for more test cases.

Note: **DO NOT HARDCODE** your methods. We will test your implemented methods using a different `main` method in which different values will be passed into these methods.

IMPORTANT : Ensure Q3.java, ShapeComparator.java
are in the folder <your email ID>/Q3
Only these files will be marked